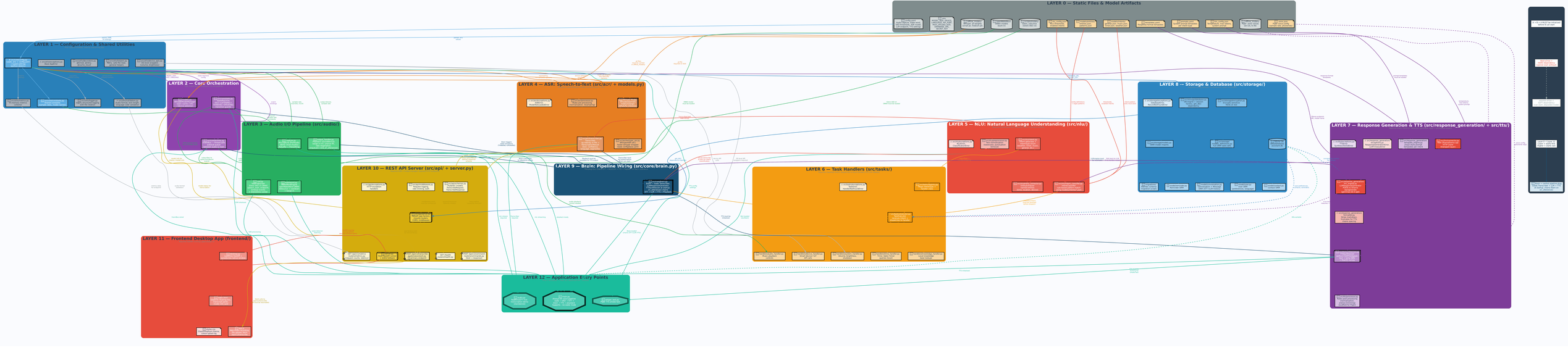

Offline Speech Recognition System (ASR) — Voice Assistant
Detailed Service Dependency & Initialization Order

Arrow A → B = A must be initialized before B can start | Bold = hard dependency | Dashed = soft/optional | Lower layers start first

LAYER 0 — Static Files & Model Artifacts

- config.yaml
Audio capture, wake word, VAD thresholds, ASR model, LLM endpoint, TTS settings
- .env
MODEL_SIZE, DEVICE, LANGUAGE, API_PORT, MAX_UPLOAD_SIZE, DATABASE_URL, SECRET_KEY
- offline_models/
Whisper .pt weights (small.pt, medium.pt)
- openWakeWord
ONNX models (built-in)
- scripts/models/silero_vad.onnx
(Silero VAD v5)
- nlu_config.json
NLU thresholds, enabled intents
- models/entities/entities.json, patterns.json
- models/intents/general.json, music.json, timer.json, weather.json
- templates.yaml
Response format templates
- prompts.yaml
System prompt templates per intent type
- llm_config.json
Temperature, max tokens, system prompt
- offline_models/
Piper .onnx voices (en-US, hi-IN)
- .onnx.json
Piper voice config (sample rate, phonemes)

LAYER 1 — Configuration & Shared Utilities

First code to initialize; used by everything else

- ⚙ src/core/config.py
Loads .env + config.yaml → typed settings (python-dotenv)
- ⚠ src/utils/exceptions.py
Base AppError
- src/utils/decorators.py
@timed, @retry, @cache_result
- src/utils/helpers.py
Retry, safe file read, general helpers
- src/utils/device_apis.py
OS-level wrappers (volume, brightness)
- src/utils/metrics.py
In-memory metrics collector
- src/core/constants.py
Shared constants (sample rates, model names)
- src/utils/logger.py
loguru structured logging with file rotation
- ✔ src/utils/validators.py
Audio format, text length, language code validation

LAYER 2 — Core Orchestration

FSM, Event Bus — foundation for all pipelines

- models.py
WhisperModelManager load / cache / info / device selection
- src/core/state.py
StateMachine IDLE→LISTENING→THINKING→SPEAKING transition guards
- src/core/events.py
EventBus — thread-safe pub/sub queue between audio & main

LAYER 3 — Audio I/O Pipeline (src/audio/)

Hardware audio access + detection services

- capture.py
MicCapture (PortAudio 16kHz int16 chunks) Rebuffer + PrerollRing
- output_handler.py
Playback (sounddevice) barge-in kill, silence fill, rate negotiation, enqueue() / end_of_utterance()
- vad.py
VadEndpointer Silero VAD v5 ONNX speech_start, endpoint, timeout events + SileroOnnx runner
- wakeword.py
WakeWordEngine openWakeWord ONNX debounce + cooldown + barge-in

LAYER 4 — ASR: Speech-to-Text (src/asr/ + models.py)

Requires loaded Whisper model; converts audio → text

- ⚠ src/asr/exceptions.py
ASRError, ModelNotLoadedError
- src/asr/processor.py
Audio pre-processing (normalisation, resampling)
- Whisper Model
(tiny|base|small|medium|large) GPU/CPU inference
- src/asr/transcriber.py
Transcriber np.ndarray or file → TranscriptionResult (text, confidence, language, segments)
- src/asr/whisper_asr.py
WhisperASR (alt.) faster-whisper wrapper reads config/base.yaml

LAYER 5 — NLU: Natural Language Understanding (src/nlu/)

Requires transcribed text; classifies intent + extracts entities

- ⚠ src/nlu/exceptions.py
NLUError, ClassificationError
- src/nlu/preprocessor.py
Text normalisation (lowercase, punctuation removal)
- src/nlu/intent.py
Intent dataclass + IntentType enum (weather, timer, music, general, etc.)
- src/nlu/entity_extractor.py
EntityExtractor named entities (time, location, device)
- src/nlu/intent_classifier.py
IntentClassifier rule-based pattern match using models/intents/*.json

LAYER 6 — Task Handlers (src/tasks/)

Requires classified intent; dispatches to local handlers (no LLM needed)

- ⚠ src/tasks/exceptions.py
TaskError, HandlerNotFoundError
- src/tasks/registry.py
Maps IntentType → handler class
- ⚡ src/tasks/executor.py
TaskExecutor receives Intent → dispatches to handler
- handlers/audio_handler.py
Music playback controls
- handlers/device_handler.py
Smart device / IoT API calls
- handlers/system_handler.py
Volume, brightness, shutdown
- handlers/time_handler.py
Time / date / timer queries (local)
- handlers/knowledge_handler.py
Local knowledge base lookups

LAYER 7 — Response Generation & TTS (src/response_generation/ + src/tts/)

Requires intent (NLU) or transcript; generates reply + synthesizes speech

- ⚠ src/tts/exceptions.py
TTSError, SynthesisFailedError
- ⚠ src/response_generation/exceptions.py
ResponseGenerationError, LLMTimeoutError
- src/response_generation/template_engine.py
Jinja2-style prompt templates per intent
- src/llm/connector.py
Raw Ollama/llama.cpp HTTP client (transport layer)
- src/response_generation/llm_engine.py
LLMResponseGenerator POSTs to Ollama /api/generate (gemma2:2b-it-qat)
- ✂ src/response_generation/formatter.py
Strips markdown, truncates for TTS, cleans spacing
- src/tts/synthesizer.py
TTSSynthesizer pyttsx3 (SAPI5/eSpeak) text → temp WAV → float32 numpy
- src/tts/processor.py
Audio post-processing (normalisation, silence trimming) [scaffold for Piper]

LAYER 8 — Storage & Database (src/storage/)

Requires config; persists conversations, users, reminders

- ⚠ src/storage/exceptions.py
DatabaseError, RecordNotFoundError
- src/storage/database.py
DB engine + session factory, get_db() dependency
- src/storage/encryption.py
Encrypts sensitive fields at rest
- src/storage/models.py
ORM model imports
- src/storage/models_definitions.py
Full ORM table defs
- SQLite DB
data/database/assistant.db
- migrations/
Alembic schema migration scripts
- model/reminder.py
Reminder ORM
- model/conversation.py
Conversation + Message ORM (STT→LLM history)
- model/user.py
User ORM (preferences, language)
- model/contact.py
Contact ORM

LAYER 9 — Brain: Pipeline Wiring (src/core/brain.py)

The plug that chains STT → NLU/LLM → TTS → Playback on each utterance

- src/core/brain.py
Brain — loads Transcriber, LLMResponseGenerator, TTSSynthesizer at startup. on_utterance() chains STT → LLM → TTS → Playback

LAYER 10 — REST API Server (src/api/ + server.py)

Requires ASR + model manager; exposes HTTP endpoints

- ⚠ src/api/exceptions.py
HTTP exception handlers
- src/api/middleware.py
Request logging, rate limiting, auth
- src/api/schemas.py
Pydantic models TranscribeRequest, TranscribeResponse
- src/api/server.py
FastAPI app factory, mounts routers, CORS, middleware
- GET /api/v1/metrics
routers/metrics.py
[request counts, latency]
- server.py
Root FastAPI runner + stdlib fallback
- POST /api/v1/transcribe
routers/assistant.py
primary endpoint
- GET /health
routers/health.py
- POST /api/v1/command
routers/commands.py
[text → response]

LAYER 11 — Frontend Desktop App (frontend/)

Requires FastAPI running; glass-design UI

- desktop.py
Launches FastAPI + pywebview native window
- index.html
Glass UI: floating mic, transcript panel, model info card
- styles.css
Glassmorphism theme, colour-splash bg
- app.js
Web Audio recording, file upload, fetch /api/v1/transcribe

LAYER 12 — Application Entry Points

Final executables — need ALL dependencies initialized

- audio.py
RealtimeSTT live-transcription demo (standalone)
- main.py
Always-on voice pipeline wake → VAD → STT → brain → TTS → playback Supports --no-wake mode
- whiper_test.py
Piper TTS smoke test

A → B = A MUST be initialized before B can start

Bold arrow = HARD dependency (cannot start without)

Dashed arrow = SOFT dependency (can start, degraded mode)

Layer 0 → Layer 12
Lower = starts first
Higher = starts last

Brain = central pipeline plug
Loads Transcriber + LLM + TTS at startup, chains them on on_utterance()

parses YAML all settings
parses .env values
metrics settings
log paths, rotation config
upload limits, allowed formats
MODEL_SIZE, DEVICE, LANGUAGE
state machine config
event bindings
sample rate, channels, chunk
output device, sample rate
shared sample rate constants
FSM must be ready for LISTENING state
model info for /status, /model-info
subscribes to speech/endpoint events
metrics data for endpoint
audio format validation
model status for transcription
FSM initialized
EventBus wired
VAD processing
wake detector listening
FastAPI must be serving before desktop window opens
fetch calls to /api/v1/transcribe (API must be reachable)
resampling, normalisation params
device selection cpu/cuda
loads .pt weights into GPU/CPU memory
.pt files must be cached in offline_models/
.pt file required on disk
ONNX model must be loaded
Silero VAD v5 ONNX must be loaded
transcribed text (raw string)
Brain triggers state transitions (THINKING→SPEAKING)
Brain subscribes to utterance events
Playback must be available for TTS output
Transcriber must be loaded before Brain can start
get_db() dependency injection
sends transcript to NLU for intent classification
device API calls
OS-level API wrappers required
TranscriptionResult defines response schema
middleware stack must be configured
request/response Pydantic models
transcription capability required
app factory used by root server.py
mic stream required
Transcriber required
mic streaming
playback ready
audio playback control needed
TTS config settings
Brain loaded (chains STT→LLM→TTS)
TTS must be initialized
NLU loaded (via Brain)
IntentType → handler mapping
classified Intent must be resolved before dispatch
entity definitions + regex patterns
thresholds, enabled intents
intent pattern rules must exist
DATABASE_URL, SECRET_KEY
LLM engine must be reachable
falls back to LLM if no handler match
Ollama endpoint URL, model name
response format templates
prompt templates must be loaded
temperature, max tokens, system prompt
voice config (phoneme map)
conversation history for context
user preferences, contacts, reminders
if no loca
matches →
DB available
TTS initialized
TTS must be working for smoke test